

The Pasqal Software Stack

START HERE

- **Documentation hub** (all sections in one place) : <https://docs.pasqal.com/>
- **Onboarding guide** : <https://docs.pasqal.com/onboarding/>
- **Community portal** (forums, tutorials, code library) : <https://community.pasqal.com/>
- **Community Slack** : <https://pasqal-community.slack.com>
- **Product news / release updates** : <https://docs.pasqal.com/productnews/>

QUANTUM SDKS

QoolQit : high-level analog algorithm development

A Python library for designing and compiling analog quantum programs for neutral-atom devices in the Rydberg Analog Model. It works in a dimensionless framework, so you don't have to specify atom positions in micrometers, laser amplitudes in MHz or pulse durations in nanoseconds. A good entry point for algorithms before hardware details.

- Full documentation : <https://pasqal-io.github.io/qoolqit/latest/>
- Get started / install : https://pasqal-io.github.io/qoolqit/latest/get_started/install/
- The QoolQit model : https://pasqal-io.github.io/qoolqit/latest/get_started/qoolqit_model/
- Applications & tutorials (QUBO, MWIS, TFIM) : <https://pasqal-io.github.io/qoolqit/latest/tutorials/>
- API reference : <https://pasqal-io.github.io/qoolqit/latest/reference/qoolqit/>
- Overview page on the docs hub : <https://docs.pasqal.com/qoolqit/>
- GitHub : <https://github.com/pasqal-io/qoolqit> · Install: `pip install qoolqit`

Pulser : pulse-level control

An open-source Python package with libraries for designing and simulating pulse sequences acting on programmable arrays of neutral atoms.

- Documentation : <https://docs.pasqal.com/pulser/>
- Conceptual introduction (recommended reading) : <https://docs.pasqal.com/pulser/programming/>
- First tutorial : <https://docs.pasqal.com/pulser/tutorials/creating/>
- ReadTheDocs mirror : <https://pulser.readthedocs.io>
- GitHub : <https://github.com/pasqal-io/Pulser> · Install: `pip install pulser`

Pulser Studio : no-code visual sequence builder

Browser-based, useful for a first lecture or lab session. It integrates with Pulser, so sequences can be designed visually and then refined programmatically.

- Documentation : <https://docs.pasqal.com/pulserstudio/>
- Application : <https://pulserstudio.pasqal.cloud/>

EMULATORS

EMU-SV (state vector) and EMU-MPS (matrix product states) are the two emulators for simulating quantum sequences. Both run locally and are fully integrated into the Pasqal cloud platform, with configurable noise models, observables and simulation parameters.

- **Local emulators overview** : <https://docs.pasqal.com/qpu-emulators/>
- **emu-sv** (exact; typically best for noiseless simulations up to roughly 25 qubits) : <https://docs.pasqal.com/emulationtools/emusv/> · `pip install emu-sv`
- **emu-mps** (tensor networks; better scalability unless extreme accuracy is needed) : <https://docs.pasqal.com/emulationtools/emumps/> · `pip install emu-mps`
- **Combined emulator site + benchmarks** : <https://pasqal-io.github.io/emulators/latest/>
- **Source code** : <https://github.com/pasqal-io/emulators>
- **Running emulators on the cloud** : <https://docs.pasqal.com/cloud/emulators-integration/>
- **Cloud emulator specs & parameters** : <https://docs.pasqal.com/cloud/emulators/>

Installing either emulator will update `puLser-core` , so always work inside a virtual environment.

CLOUD & EXECUTION

- **Pasqal Cloud portal** (accounts, jobs, results) : <https://portal.pasqal.cloud/>
- **Cloud documentation** : <https://docs.pasqal.com/cloud/>
- **Quick start** : <https://docs.pasqal.com/cloud/quick-start/>
- **pasqal-cloud Python SDK** : <https://github.com/pasqal-io/cloud-sdk> · <https://pypi.org/project/pasqal-cloud/> · `pip install pasqal-cloud`
- **Third-party access** (Google Cloud, Azure, OVHcloud, Scaleway) : <https://docs.pasqal.com/third-party-cloud-providers/>
- **Offers and pricing** : <https://portal.pasqal.cloud/offers>

APPLICATION LIBRARIES

QUBO solver

A Python library for solving Quadratic Unconstrained Binary Optimization problems on a neutral-atom QPU, with classical (tabu search, simulated annealing), quantum (variational algorithms, quantum adiabatic algorithm) and hybrid approaches.

- Documentation : <https://docs.pasqal.com/applicationsolvingtools/qubo/>
- Gentle introduction tutorial : <https://docs.pasqal.com/applicationsolvingtools/qubo/tutorial/00-a-tour-of-qubo/>
- Solver walkthrough : <https://docs.pasqal.com/applicationsolvingtools/qubo/tutorial/02-qubosolver/>
- Configuration reference : <https://docs.pasqal.com/applicationsolvingtools/qubo/content/config/>
- API documentation : <https://pasqal-io.github.io/qubo-solver/latest/>
- GitHub : <https://github.com/pasqal-io/qubo-solver> · Install: `pip install qubo-solver` (imports as `qubosolver`)

Related application tools

- Solving MIS via qubo-solver : <https://docs.pasqal.com/applicationsolvingtools/mis/advanced tutorial 3 - Using qubo-solver/>
- QEK : quantum graph machine learning : <https://docs.pasqal.com/applicationsolvingtools/qek/>

Links verified September 2026. Pasqal documentation is updated frequently : check docs.pasqal.com for the latest.